CS-4063: Natural Language Processing

Assignment 1: Language Modeling for Urdu News Articles

Due Date: 15 Feb 2026 Time 11:59 PM

Introduction

This assignment focuses on building a complete end-to-end Natural Language Processing pipeline for the Urdu language using real-world BBC Urdu news data.

Students will scrape news articles, normalize and preprocess the dataset, implement custom tokenizer, stemmer, and lemmatizer, and finally generate **BBC-style Urdu news** articles using Bigram and Trigram language models.

The assignment is divided into two main parts:

- **Part 1:** Dataset collection and preprocessing
- **Part 2:** Automated Urdu news article generation

All NLP components must be implemented from scratch without using pretrained NLP libraries.

Learning Outcomes

After completing this assignment, students will be able to:

- Scrape structured real-world Urdu datasets
- Design data pipelines for low-resource languages
- Handle Unicode normalization challenges
- Implement custom linguistic preprocessing tools
- Train statistical language models
- Generate coherent Urdu articles
- Evaluate NLP systems

PART 1 - BBC Urdu Dataset Collection and Preprocessing

1 Description

In this part, students will collect BBC Urdu news articles and prepare a clean and structured dataset by applying normalization techniques and implementing custom preprocessing tools. The resulting processed dataset will be reused directly in Part 2 for language model training and text generation tasks.

2 Dataset Collection and Storage Format

- Students must scrape news articles from: **https://www.bbc.com/urdu**
- Students must scrape: 200 – 300 complete BBC Urdu News Articles
- Students must store mandatory article metadata in a JSON file with following structure:
    - Each article must be numbered
    - Article number must be unique
    - Article number must match the corresponding article context in .txt files
    - Article body must NOT be included in JSON
    - Example of JSON is given in the notebook
- Students must create two mandatory text files:
    1. **raw.txt**

    This file must contain:

    - All scrapped article bodies in a single text file
    - Raw content only with no cleaning or normalization applied
    - One article per block
    - Each article must start with its corresponding article number
    - Formatting must follow the example provided in the notebook

    2. **cleaned.txt**

    This file must contain:

    - All articles in a single text file
    - Fully Processed and cleaned data
    - Normalized Urdu text
    - All noise removed
    - Proper sentence segmentation applied
    - Custom Linguistic Processing applied
    - Data formatted and ready for language model training
    - Article numbering must exactly match raw.txt and the JSON Metadata

3 Data Cleaning and Normalization

This phase addresses the challenges of Urdu text preprocessing. Students perform normalization by removing diacritics, eliminating unwanted noise such as foreign language text and web artifacts, and ensuring consistent Unicode representation. Sentence segmentation is applied to transform unstructured text into linguistically meaningful units suitable for modeling.

3.1 Diacritics Removal

Urdu text often contains diacritics, which represent pronunciation but are inconsistently used in real-world data. Since most Urdu writing omits diacritics, their presence can create multiple

forms of the same word and increase data sparsity. Removing diacritics ensures uniform word representation across the dataset.

Example:
عَلِم → علم

This normalization allows the language model to treat both forms as the same word. Refer to the Jupyter Notebook for implementation details.

3.2 Noise Removal

Scraped web data contains various types of noise that are not part of the actual news content. Noise removal ensures that only meaningful Urdu text remains in the dataset.

Noise includes URLs, emojis, navigation text, English sentences, and Roman Urdu. These elements are removed because they do not contribute to Urdu language modeling and can distort word distributions.

Example:
Before cleaning:
نیوز بریکنگ Visit www.bbc.com 😊

After cleaning:
نیوز بریکنگ

The notebook demonstrates how regex-based rules are applied to identify and remove such noise.

3.3 Removal of Non-Urdu Text

BBC Urdu pages may contain embedded English phrases, Roman Urdu, or metadata-related text. Since the goal is to model pure Urdu language structure, non-Urdu scripts are filtered out.

Example:
یہ رپورٹ breaking news کے بارے میں ہے

After cleaning:
یہ رپورٹ کے بارے میں ہے

Script-based filtering logic is explained in the Jupyter Notebook.

3.4 Sentence Segmentation

Raw Urdu text is often written in long paragraphs without clear sentence boundaries. Sentence segmentation divides text into meaningful sentences, which is essential for training language models and ensuring grammatical generation. Segmentation is performed using Urdu punctuation marks such as full stop, question mark, and exclamation mark.

Example:
Before segmentation:
ہوئے متاثر لوگ ہوئی بارش میں پاکستان

After segmentation:
ہوئی۔ بارش میں پاکستان
ہوئے۔ متاثر لوگ

Sentence boundary rules and their implementation are documented in the notebook.

3.5 Whitespace and Formatting Normalization

During scraping, extra spaces, line breaks, and inconsistent formatting are common. These issues can interfere with tokenization and word frequency counts. Normalization ensures consistent spacing and clean sentence structure.

Example:
Before cleaning:
ہوئی بارش میں پاکستان

After cleaning:
ہوئی بارش میں پاکستان

This step ensures that tokens are correctly identified during further processing.

3.6 Custom Linguistic Processing

- **Custom Urdu Tokenizer**: Implement a tokenizer for word boundaries, punctuation, postpositions, and numbers (replace numbers with <NUM>). Details and examples are in the Jupyter Notebook.
- **Custom Urdu Stemmer**: Implement a stemmer to remove common suffixes and obtain root forms. Rules and examples are in the notebook.
- **Custom Urdu Lemmatizer**: Implement a lemmatizer for plural and gender normalization. Mapping rules are provided in the notebook.

4 Output of Part 1

By the end of Part 1, students must produce the following files:

- **Metadata.json:** Metadat for 200-300 BBC Urdu articles (no article text), with consistent and unique article numbering.
- **raw.txt**: All scraped article bodies in a single file, unprocessed, one article per block, numbered.
- **cleaned.txt:** A single file containing fully preprocessed BBC Urdu articles as normalized, sentence-segmented, lemmatized, stemmed tokens (numbers replaced with <NUM>), ready for language model training. Article numbering must match raw.txt and Metadata.json.

Part 2 - BBC Style Urdu News Article Generation

1 Language Model Training

In this part, students will train statistical language models using the preprocessed dataset produced in **Part 1**.
Students must implement the following language models:

- **Bigram Language Model**
- **Trigram Language Model**
- **Unigram Language Model (for backoff and evaluation only)**

Both the Bigram and Trigram models must be trained on the processed token-level data obtained from the cleaned dataset. Probability estimation must be based on token sequences generated by the custom preprocessing pipeline. The Unigram model must be implemented to support the backoff mechanism and perplexity evaluation and is not required to be used for standalone article generation.

2 Smoothing Technique

To handle unseen n-grams and avoid zero probability issues, students must apply smoothing during language model training.
Allowed smoothing methods:

- **Laplace (Add-One) Smoothing**
- **Add-k Smoothing** (where k is a small positive constant)

3 Article Generation System and Interface

Students must implement an article generation system that produces Urdu news text using the trained language models.
The system must support:

- Language model selection (Bigram or Trigram)
- Seed prompt input
- Automatic article generation

Proper Right-to-Left (RTL) display of Urdu text must be ensured.

Bonus (Optional)
A GUI-based interface (e.g., using Tkinter, Streamlit, or any other framework) may be implemented for additional credit. A console-based interface is sufficient for core requirements.

4 Seed Prompt and Generation Constraints

The seed prompt must contain 5–8 Urdu words, and single-word prompts are not allowed.

Example of a valid seed prompt is: میں شرح کی مہنگائی میں پاکستان
Example of an invalid seed prompt is: پاکستان.

Each generated article must have a minimum length of 200 words, a target length of up to 250 words, and contain at least 5 sentences. Generation must stop at a maximum of 300 words, and forced termination must be applied if an End-of-Sentence (EOS) token is not produced.

The generated text must be original and must not reproduce complete sentences from the training dataset. Memorized or duplicated content is strictly prohibited, and systems producing copied output will be penalized.

Students must generate the following outputs: **three complete Urdu news articles using the Bigram language model**, **three complete Urdu news articles using the Trigram language model with backoff (Trigram → Bigram → Unigram),** and **five Urdu news headlines overall**. The backoff model means that when the trigram context is not available, the system automatically uses the bigram model, and if needed, falls back to the unigram model. All generated content must follow BBC-style Urdu news language and structure, including formal tone, coherent sentence flow, and proper paragraph formatting.

5 Evaluation And Analysis

Students must evaluate the generated text both qualitatively and comparatively.

1. **Evaluation:**
   Each generated article must be evaluated based on:
   - **Fluency**
   - **Grammatical correctness**
   - **Coherence and readability**
   - **Perplexity score**
2. **Comparative Analysis:**
   Generation results must be compared across different preprocessing pipelines and language models:
   - **Raw text pipeline**
   - **Cleaned text pipeline**
   - **Bigram language model**
   - **Trigram language model with backoff**

The analysis should highlight differences in text quality, coherence, and generalization performance and perplexity values.

6 Submission Guidelines

1. **Report Requirements:** The report should briefly cover the project overview, data collection and preprocessing methods, language model training with smoothing techniques, the article generation process including constraints, evaluation and comparison of results, and a short conclusion highlighting challenges and key findings.
   Formatting Guidelines:
   - Font: Times New Roman
   - Font Size: 12
   - Line Spacing: 1.5
   - Proper headings and subheading for each section
   - Length: Around 2-3 pages of content including plots and discussion

2. **Submission Format:** Students must submit a **zip folder** named: **i23-XXXX_Assignment1_DS-X.zip** where **XXXX** is your **student ID** and **X** is your **section.**
   The zip folder must contain:
   - The Jupyter notebook named **i23-XXXX_Assignment1_DS-X.ipynb**
   - Metadata JSON file
   - Two text files (raw.txt and cleaned.txt)
   - The report document (PDF)

3. **Plagiarism:** Any form of plagiarism will result in zero marks. Ensure all code and report content is your own work.

4. **Outputs:** Your notebook must include all outputs such as code execution results, tables, and plots. Missing outputs will result in loss of marks.

Rubric:

The following rubric will be followed for marking of the assignment and evaluations.

Part 1: Dataset Collection & Preprocessing (30 Marks)

1. Dataset Collection & Metadata Management (6 Marks)

- Scraping 200–300 complete BBC Urdu articles (2 Marks)
- Correct JSON metadata structure with unique numbering and no article body (2 Marks)
- Consistency between JSON, raw.txt, and cleaned.txt numbering (2 Marks)

2. Raw Data File (raw.txt) Compliance (4 Marks)

- All article bodies included without any preprocessing (2 Marks)
- Correct formatting with one article per block and proper numbering (2 Marks)

3. Data Cleaning & Normalization (10 Marks)

- Diacritics removal correctly implemented (2 Marks)
- Noise removal including URLs, emojis, and web artifacts (2 Marks)
- Removal of non-Urdu, Roman Urdu, and English text (2 Marks)
- Sentence segmentation using Urdu punctuation (2 Marks)
- Whitespace and formatting normalization (2 Marks)

4. Custom Linguistic Processing (10 Marks)

- Custom Urdu tokenizer including word boundaries, punctuation, and <NUM> handling (4 Marks)
- Custom Urdu stemmer with correct suffix handling (3 Marks)
- Custom Urdu lemmatizer for plural and gender normalization (3 Marks)

Part 2: BBC-Style Urdu News Article Generation (30 Marks)

1. Language Model Implementation (10 Marks)

- Unigram language model implemented for backoff and evaluation (2 Marks)
- Bigram language model implementation (4 Marks)
- Trigram language model with backoff from trigram to bigram to unigram (4 Marks)

2. Smoothing & Probability Estimation (4 Marks)

- Correct application of Laplace or Add-k smoothing (2 Marks)
- Proper handling of unseen n-grams with no zero probabilities (2 Marks)

3. Article Generation System & Constraints (8 Marks)

- Model selection and valid seed prompt handling with 5–8 Urdu words (2 Marks)

- Length constraints and EOS handling with 200–250 words and maximum 300 words (2 Marks)
- Correct article generated as output from the models with backoff, outputs with three complete articles (2 Marks)

4. Quality of Generated Content (5 Marks)

- Fluency and grammatical correctness (2 Marks)
- Coherence, readability, and BBC-style formal tone (2 Marks)
- Originality with no memorized or copied sentences (1 Mark)

5. Evaluation & Comparative Analysis (3 Marks)

- Perplexity calculation and reporting (1 Mark)
- Comparison between raw and cleaned preprocessing pipelines (1 Mark)
- Comparison between Bigram and Trigram models with backoff (1 Mark)

Bonus: User Interface (UI) Implementation

1. Basic UI Implementation (2 Marks)

- Functional Frontend-based interface for article generation (2 Marks)

2. UI Functionality & Usability (2 Marks)

- Language model selection option (Bigram or Trigram) (1 Mark)
- Seed prompt input and generation trigger through the UI (1 Mark)

3. Presentation & RTL Handling (1 Mark)

- Proper Right-to-Left (RTL) display of Urdu text with readable formatting (1 Mark)

Report and Documentation (10 Marks)

- Report is complete, well-structured, and properly formatted according to the guidelines (2 Marks)
- Clear explanation of dataset collection and preprocessing approach (2 Marks)
- Description of language model implementation and smoothing/backoff method (2 Marks)
- Explanation of article generation process and applied constraints (2 Marks)
- Evaluation discussion with basic analysis and comparison of results (2 Marks)

***** Good Luck *****